

# Selective supplementary experiments

## Prediction decorrelation and step-size robustness

1 October 2026

### Contents

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>1. Recommendation and strict scope</b>                                 | <b>2</b>  |
| 1.1 Why these additions are worth more than another broad sweep . . . . . | 2         |
| 1.2 Mandatory spectral reanalysis comes first . . . . .                   | 3         |
| 1.3 Audit of the supplementary claims . . . . .                           | 3         |
| <b>2. Freeze the common setup</b>                                         | <b>3</b>  |
| <b>3. Inventory and recovery before any run</b>                           | <b>4</b>  |
| 3.1 Build one manifest . . . . .                                          | 4         |
| 3.2 Recovery ladder and what each level costs . . . . .                   | 4         |
| 3.3 Small source check before a large export . . . . .                    | 5         |
| <b>4. S1: does width improve prediction decorrelation?</b>                | <b>5</b>  |
| 4.1 Exact question and observables . . . . .                              | 5         |
| 4.2 Targets and retained window . . . . .                                 | 5         |
| 4.3 Estimator: normalize first, then average . . . . .                    | 5         |
| 4.4 Lag range and numerical cases . . . . .                               | 6         |
| 4.5 What to plot and calculate . . . . .                                  | 6         |
| 4.6 Interpretation and caption . . . . .                                  | 7         |
| <b>5. S2: does the deep width effect persist at a smaller step size?</b>  | <b>7</b>  |
| 5.1 Scope and why only endpoints . . . . .                                | 7         |
| 5.2 Equal algorithmic duration . . . . .                                  | 7         |
| 5.3 Estimator and displayed comparison . . . . .                          | 8         |
| 5.4 What the result supports . . . . .                                    | 8         |
| <b>6. Reuse the predictive-score result as a table</b>                    | <b>9</b>  |
| <b>7. Diagnostics: enough to assess the new claims</b>                    | <b>9</b>  |
| 7.1 Numerical and bookkeeping requirements . . . . .                      | 9         |
| 7.2 Per-observable trace diagnostics . . . . .                            | 10        |
| 7.3 What not to require . . . . .                                         | 10        |
| <b>8. Build sequence and acceptance checks</b>                            | <b>10</b> |
| 8.1 Minimal implementation . . . . .                                      | 10        |
| 8.2 Source adapter acceptance . . . . .                                   | 11        |
| 8.3 Publication checks . . . . .                                          | 11        |
| <b>9. If new trajectories are actually necessary</b>                      | <b>11</b> |
| 9.1 Decide the scope before launching . . . . .                           | 11        |
| 9.2 Exact replacement-run settings . . . . .                              | 12        |
| 9.3 Runtime and storage preflight . . . . .                               | 12        |
| <b>10. Final paper placement and stopping rule</b>                        | <b>13</b> |
| <b>References</b>                                                         | <b>14</b> |

# 1. Recommendation and strict scope

The revised main spectral figure measures the theorem-defined statistic and its empirical coverage for all displayed cutoffs, marking the admissibility threshold 2. The main loss figure measures training-loss autocorrelation under a fixed discrete kernel. The first concerns the local domain directly; the second is compatible practical evidence, not a functional-inequality test. The most useful remaining questions are whether the second observation extends to predictions and whether it survives a nearby sampler step size.

**Revision 2: first complete the mandatory spectral reanalysis in the main figure, then consider at most two supplementary figures in this priority order:**

| Priority   | Analysis                                                  | What it adds                                                                | Default work                                                                                               |
|------------|-----------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| S1         | Held-out prediction autocorrelation                       | Tests whether the width benefit extends beyond the training-loss observable | Reanalyze eight probability traces already prescribed in every dynamics run                                |
| S2         | Deep endpoint width comparison at two step sizes          | Algorithm-robustness check, not theorem validation                          | Reuse existing eligible $h = 0.005$ and $h = 0.01$ trajectories only; omit if expensive recovery is needed |
| Table only | Prior/posterior held-out predictive score and diagnostics | Establishes learning despite nearly unchanged hidden spectral norms         | Reuse the completed predictive check and trace diagnostics                                                 |

These are separate questions. S1 can succeed or fail independently of S2. Neither estimates a PI/LSI constant, integrated autocorrelation time, continuous-time rate, or empirical scaling exponent. Neither requires a new posterior target in the preferred route.

The original campaign specification required these observables and step-size runs, but the supplied figures and narrative do not prove that every necessary array is still available. Treat their availability as something to inspect, not something established by this document. Raw campaign archives were not available while preparing this plan; no new sampling or scientific reanalysis has been performed here.

## 1.1 Why these additions are worth more than another broad sweep

Prediction decorrelation addresses an immediate limitation of the main result: a chain may move quickly in training loss while useful predictions remain correlated. Function-space behavior is central to BNN posterior studies, including the predictive comparisons of Izmailov et al. and the function-space chain assessment of Sharma et al. [R1, R2] This motivates checking probabilities; their studies do not imply a favorable width effect in this model.

A nearby step size addresses a distinct question about algorithm dependence. The existing analysis deliberately makes a claim about one discrete kernel. Showing the same width ordering at a second step size broadens that empirical statement without estimating a diffusion limit. Do not impose the old integrated-time comparison gate on this new estimand.

An extra depth, real dataset, or sample-size sweep would require new targets and substantial reference sampling. Those extensions are scientifically reasonable but offer less evidence per unit of work at this stage. Do not add them to this plan. Recovering the existing statewise coverage curve

takes priority over generating more nominal samples; the revised analysis assesses cutoffs near the theorem threshold.

## 1.2 Mandatory spectral reanalysis comes first

Recover the 196,608 unrounded statewise spectral norms (or recompute them from the archived reference matrices). Use  $T = \|W_2\|_{\text{op}}/\sqrt{m}$ , not the former  $T/2.5$ . Confirm the existing setup has  $r_0 = 0$  and  $\sigma = 1$ , so  $a_* = 2$ . Follow the companion cleanup guide for the quantile panel, full empirical-coverage panel, and exact per-target counts above 2.

This is part of the main result, not an additional sampling experiment or a third supplementary figure. It replaces a selected generous cutoff by the theorem’s threshold and the entire measured coverage function. Nonzero mass above 2 does not violate the strict-cutoff result: its conclusion is for each fixed  $a > 2$  as width grows. Do not claim that the fraction above 2 must decrease to zero.

Keep the prior quantile comparison in the existing appendix table. No added cutoff sweep requires chains: every empirical coverage value comes from reclassifying the same unrestricted samples. No new widths, depths, or sample sizes are recommended for the current scope.

## 1.3 Audit of the supplementary claims

S1’s eight probabilities are prediction observables fixed in the original logging protocol; none is selected because it gives a favorable width trend. Their ACFs measure a practical aspect of posterior sampling. They are not the spectral-domain statistic and do not directly verify an LSI or PI inequality. S2 varies an algorithm parameter,  $h$ , rather than a theorem parameter. Its role is numerical robustness of an observed width ordering. This distinction belongs in the methods and prevents either supplement from being called direct theorem validation.

The chosen probability indices, lag grids, chain counts, percentile summaries, and diagnostic thresholds are justified experimental or numerical choices. They are not mathematical constants derived from the proof. Keep them explicit, fixed across comparisons, and separate from theorem-defined reference lines. No arbitrary positive spectral margin or inferred Gaussian-rate benchmark is introduced anywhere in these supplements.

## 2. Freeze the common setup

Use the exact existing targets: binary classification,  $n = 128$ ,  $d = 32$ ,  $\tanh$ ,  $\sigma = 1$ , summed cross-entropy, all parameter blocks sampled, and the original three dataset/prior-center replicate IDs 0, 1, 2. Shallow widths are 64, 256, 1024, 4096; deep widths are 32, 64, 128, 256. The first-layer prior-center bank and datasets are shared across widths within a replicate. Do not reinitialize the centers or regenerate the dataset to obtain a favorable result.

Use the existing centered-logit convention:

$$p_j(\theta) = \Pr(y = 1 \mid x_j, \theta) = \text{sigmoid}(\sqrt{2}f_\theta(x_j)).$$

The same posterior, input preprocessing, parameter coordinates, and Metropolis-adjusted Gaussian-preserving Langevin implementation must be used at each width. A different implementation, loss normalization, preconditioner, or parameter rescaling is a different experiment.

Maintain the original reference audit and the old `dynamics_unresolved` record. New supplementary eligibility is scoped to finite-lag, fixed-kernel observables. A failure of a previous integrated-time or continuous-time precision criterion is neither automatic exclusion nor automatic clearance: inspect the actual scalar traces and apply the rules below.

### 3. Inventory and recovery before any run

#### 3.1 Build one manifest

Make one row per target, step size, chain, and segment. Record:

- Architecture, width, replicate ID, chain ID,  $h$ , sampler name and code commit.
- Dataset, test set, prior-center bank, model configuration, and likelihood-normalization hashes.
- Original segment role: burn-in, calibration, production, refinement, endpoint validation, or restart.
- Retained transition range, storage stride, actual array shape, dtype, file path, and SHA-256.
- Presence of  $V$ , the eight held-out probabilities or their logits, rejection flags, proposal counters, and restart state/RNG.
- Whether an audited reference initialization exists for this exact target.

Segment names do not determine scientific eligibility. For example, a final  $h = 0.01$  trajectory saved under a refinement directory can be the correct main-analysis trajectory. Match metadata and transition IDs; do not count the same trajectory twice because two exports refer to it.

#### 3.2 Recovery ladder and what each level costs

| Available material                                                               | Action                                                      | New posterior simulation?                                                              |
|----------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Saved scalar probabilities at every retained step                                | Read the original arrays and map the fixed test indices     | No; cheap reanalysis                                                                   |
| Saved logits at every retained step                                              | Apply the exact sigmoid convention in float64               | No; cheap reanalysis                                                                   |
| Full dynamics states at every retained step                                      | Evaluate the eight fixed test points in batches             | No new states; forward-pass work may be substantial                                    |
| Accepted-state probabilities plus every accept/reject flag and the initial value | Reconstruct repeats exactly                                 | No; verify transition convention                                                       |
| Checkpoints with complete RNG and exact original implementation                  | Replay the original trajectories and record missing scalars | Same scientific realization; computationally comparable to rerunning those transitions |
| Sparse checkpoints without reproducible RNG                                      | Cannot recover intermediate states                          | A new trajectory is required for unit-stride ACF                                       |
| Reference sampler states only                                                    | Use them for posterior expectations, not Langevin ACF       | Cannot answer the requested dynamics question                                          |

Interpolation is not recovery. Repeating each sparse saved probability until the next checkpoint invents a new process and distorts autocorrelation. Equilibrium posterior samples from elliptical slice sampling do not become Langevin samples by assigning them a step size.

If only a regularly thinned dynamics series survives and its stride is at most 5, it can support an explicitly coarsened supplementary analysis: use the same stride at every compared width, label the x axis in original iterations, and state the stride in the caption. Compute only available lags; do not draw fabricated intermediate values. A requested table lag not divisible by that stride is not available, not an interpolated value (for example, lag 25 for stride 2). Prefer the unit-stride route because the original campaign was designed to save it. If strides are incompatible or too coarse to resolve lags 25–200, use exact replay or the capped fallback in Section 9.

### 3.3 Small source check before a large export

For one chain at each endpoint width in each architecture, compare stored probabilities/logits against a recomputation from the corresponding saved state at five available checkpoints. Verify the original test-point indices and  $\sqrt{2}$  factor. Use absolute tolerance  $10^{-11}$  plus relative tolerance  $10^{-9}$  in float64; investigate any discrepancy rather than rounding it away.

At a rejected move, every saved observable must repeat its previous state value. A first retained move needs the last discarded state to check this rule. Confirm that stored values are post-transition states. Check the last exported  $V$  against the last saved parameter state as in the accepted main analysis.

## 4. S1: does width improve prediction decorrelation?

### 4.1 Exact question and observables

Measure dependence in the predicted probability of class 1 at the **eight originally specified held-out points**, with zero-based indices

$$J = (0, 128, 256, 384, 512, 640, 768, 896).$$

Use every one of these points. Do not select test points by label, confidence, variance, agreement with the width trend, or apparent slowness. The same test points are used across widths within a data replicate; different replicates contain different datasets.

The prediction ACF is a simple observable-level measurement. It asks whether repeated sampler states provide less temporally dependent values of these probabilities as width grows. It does not establish accurate exploration of every possible network prediction or posterior mode.

### 4.2 Targets and retained window

Preferred scope: all 24 existing targets at  $h = 0.01$ , with four chains per target. Use the **same final 102,400 retained transitions per chain used for the main loss figure**. Use explicit original transition IDs, including the later window for the three targets whose archived chains ran longer.

The main figure's passed diagnostics for  $V$  do not imply passed diagnostics for each probability. Apply the per-observable checks in Section 7 to the chosen window before aggregation.

Export one NPZ per target with numeric arrays:

- probabilities: shape (4, 102400, 8), float64.
- V: shape (4, 102400), float64, copied from the matched loss export.
- iteration: shape (4, 102400), integer, consecutive within each chain.
- test\_indices: shape (8, ), equal to  $J$ .

Store target metadata in a JSON sidecar. Use `allow_pickle=False` when loading arrays. The probability array is about 26.2 MB per target before compression, or about 629 MB for all 24 targets. Stream per-target calculations if memory is limited; there is no need to load all targets simultaneously.

### 4.3 Estimator: normalize first, then average

For chain  $c$ , test point  $j$ , and lag  $k$ , let  $z_{c,t,j} = p_j(\theta_{c,t}) - \bar{p}_{c,j}$ . Calculate

$$\hat{\rho}_{c,j}(k) = \frac{\sum_{t=0}^{N-k-1} z_{c,t,j} z_{c,t+k,j}}{\sum_{t=0}^{N-1} z_{c,t,j}^2}.$$

This is the same unadjusted normalized estimator used for the accepted loss ACF. Calculate it independently for each of the 32 chain/point series. Do not concatenate chains and do not subtract a pooled mean. Use zero-padded FFT correlation with padding at least  $2N - 1$ ; no circular wraparound and no  $N/(N - k)$  adjustment.

For target replicate  $r$ , report

$$\hat{\rho}_r^{\text{pred}}(k) = \frac{1}{8} \sum_{j=1}^8 \frac{1}{4} \sum_{c=1}^4 \hat{\rho}_{r,c,j}(k).$$

This is **mean predictive autocorrelation across eight fixed points**, not the autocorrelation of their averaged probabilities. The latter could hide dependence through cancellation, and would answer a different question. Normalizing each series first gives every point equal weight, regardless of its probability variance.

At each lag, take the median of the three replicate curves. The shaded range is their minimum to maximum. Keep each point’s four-chain average ACF in a sidecar so heterogeneous outcomes remain inspectable.

## 4.4 Lag range and numerical cases

Compute lags 0–400 first. If any width’s median mean-predictive ACF has absolute value greater than 0.10 at the last lag, extend the calculation to 800, then 1600, then 3200 until this endpoint condition no longer holds. Use the same final maximum lag in both architecture panels. This uses the same existing samples; it is not an instruction to run more chains.

The threshold is a display rule, not a relaxation-time estimate. Inspect the full curves for late rebounds or substantial sign changes as well; if the endpoint alone conceals such structure, display the full computed 3200-lag range and describe it. Do not clip negative values or assume every curve is monotone.

Probabilities must be finite and within  $[0,1]$ . If a chain’s standard deviation is below  $10^{-10}$  in probability units, mark the point `numerically_degenerate` and inspect the original logits for saturation or an export error. The threshold is a numerical safeguard. Do not replace its ACF with zeros or silently average only the remaining points. If genuine degeneracy prevents the prescribed eight-point summary, report the affected point and use the complete per-point table; do not advertise an eight-point curve based on fewer points.

## 4.5 What to plot and calculate

**One two-panel figure:** “(a) Shallow” and “(b) Deep”; x label “Lag (sampler iterations)”; y label “Mean prediction autocorrelation.” Use the cleanup guide’s width colors, line thicknesses, fonts, and replicate-range shading. Show all four widths per architecture in the preferred reuse route. For a longer lag horizon, choose five evenly spaced major ticks and marker positions at one-eighth increments of the horizon; omit the common lag-zero marker.

Export a compact table at lags 25, 50, 100, 200. For each architecture, replicate, and test point, calculate the paired difference

$$D_{r,j}(k) = \hat{\rho}_{r,m_{\min},j}(k) - \hat{\rho}_{r,m_{\max},j}(k).$$

Positive values mean lower autocorrelation at the widest width. Also report the replicate-specific difference after averaging the eight points and the number of positive point differences out of eight in each replicate. These counts are descriptive: test points in a chain are dependent, and 24 point/replicate entries are not 24 independent experimental replicates. Do not attach a binomial p-value or pretend the three data replicates provide a precise population confidence interval.

Do not add an ESS-versus-width plot; it would substantially repeat the autocorrelation information. Keep ESS in the diagnostic table.

## 4.6 Interpretation and caption

**Favorable pattern:** the wider deep-network curves lie below the narrower ones at meaningful finite lags, and most fixed test points show the same direction. This would extend the empirical observation from loss to predictive observables. No particular magnitude is promised.

**Flat pattern:** loss decorrelation improves but these probabilities do not show a clear width benefit. This still adds useful scope to the paper. Report the flat result without trying other test points.

**Mixed pattern:** give the mean curve and the per-point table, and describe the effect as observable-dependent. Do not title the figure “faster predictive mixing” if the average is driven by a few points.

Neutral caption template, usable before knowing the outcome:

**Prediction autocorrelation across widths.** We compute autocorrelation separately for class-1 probabilities at eight fixed held-out inputs, average over four chains and the eight inputs within each replicate, and show the median and range across three dataset/prior-center replicates. Each chain contributes the same final 102,400 retained transitions used in the loss analysis, with  $h = 0.01$ . Colors identify width. This tests whether the width pattern observed for training loss also appears in predictive observables.

Append one factual outcome sentence after inspecting the result. Do not put a hoped-for positive result into the caption in advance.

## 5. S2: does the deep width effect persist at a smaller step size?

### 5.1 Scope and why only endpoints

Use only the deep architecture at widths **32 and 256**, all three existing data replicates, and four chains per target. Compare  $h = 0.01$  **and**  $h = 0.005$ . This is six posterior targets and twelve target/step-size combinations, all corresponding to models already in the main campaign.

The strongest main width effect is deep, so testing its endpoint contrast is more valuable than repeating all intermediate widths. Do not add shallow runs or a third step size. The observable is **training loss**  $V$ , which is already the main figure’s estimand and is recorded in every relevant trajectory. S2 does not depend on S1 showing a favorable result.

S2 is a reuse-only optional check. The original campaign planned endpoint half-step trajectories, but determine their actual length and metadata from the manifest. Do not assume a file named “halfstep” has  $h = 0.005$ : earlier comparisons may instead refer to 0.02 versus 0.01.

### 5.2 Equal algorithmic duration

For S2, use the last  $H = 512$  **units of algorithmic duration per chain** after the applicable discard segment. At  $\sigma = 1$  this means:

| Step size | Retained transitions per chain | Lag index at $t = 0.5$ |
|-----------|--------------------------------|------------------------|
| 0.01      | 51,200                         | 50                     |
| 0.005     | 102,400                        | 100                    |

Recompute the  $h = 0.01$  endpoint ACF using this matched window; do not reuse its 102,400-transition main-figure curve as if the analysis windows were equal. Keep a separate S2 export so the main figure is not changed.

Use  $t = kh$  on the x axis. Here  $t$  is only the accumulated step parameter of the discrete algorithm. It is not a fitted continuous-time relaxation rate. Comparing at the same raw lag  $k$  would conflate halving the step size with halving the accumulated step parameter.

If the smallest complete available duration across eligible combinations is less than 512 but at least 256, a no-new-sampling fallback may use that common duration, rounded down to a multiple of 128. Apply it to every combination, report it in the caption, and run all diagnostics again. Do not make each panel use an unreported different window. If complete duration is below 256, retrieve additional saved scalar segments if available; otherwise omit S2. Do not launch new smaller-step trajectories or expensive full-run replay solely for this optional check.

### 5.3 Estimator and displayed comparison

Use the ordinary per-chain  $V$  ACF formula from S1 and the main analysis, then average four chains within each replicate. No probability averaging is involved in S2.

Display a two-panel figure: “**(a)**  $h = 0.01$ ”, “**(b)**  $h = 0.005$ ”. Each panel has two solid curves, widths 32 and 256, in their established colors and markers. Show a shared two-entry width legend below the figure. Use the same y scale in both panels, ordinarily  $[-0.08, 1.02]$ . The y label is “Loss autocorrelation”; the x label is “Algorithmic lag  $kh$ ”. No additional panel header is needed.

Use  $t = 0$  to 2 initially, with ticks 0, 0.5, 1, 1.5, 2. Calculate at each step size’s native integer lags; no interpolation is needed because  $h = 0.005$  resolves every  $h = 0.01$  grid point. If any displayed median has  $|\rho(2)| > 0.10$ , extend both panels to  $t = 4$ , then  $t = 8$ . A persistent long tail is a result, not a request to fit an exponential or suppress the tail.

The comparison table uses  $t \in \{0.25, 0.5, 1, 2\}$ . For each step size and replicate calculate

$$D_{r,h}(t) = \hat{\rho}_{r,32,h}(t/h) - \hat{\rho}_{r,256,h}(t/h).$$

All listed  $t/h$  are integers. Use exact integer indexing after checking numerical rounding tolerance. Report the three differences and their median. Compare the direction and practical size of the endpoint width effect across the two kernels. Do not require their entire ACF curves or their differences to match within an arbitrary percentage, especially near zero.

### 5.4 What the result supports

If the widest model has lower loss ACF at both step sizes across the three replicates at the informative lags, state that the width ordering persists under this step-size change. It remains an observation about two finite-step kernels. If ordering changes, report the dependence and keep the main claim explicitly tied to  $h = 0.01$ .

A flat difference at late lags, when both correlations are nearly zero, is not evidence against the early-lag effect. Conversely, a favorable median does not override a large adverse result in a replicate; show all three paired differences in the sidecar/table.

Neutral caption template:

**Step-size sensitivity of the deep width comparison.** Training-loss autocorrelation is compared at widths 32 and 256 for two step sizes of the same adjusted Langevin kernel. The horizontal axis is the discrete algorithmic lag  $kh$ . Each chain contributes the final 512 units of retained algorithmic duration; curves show the median and range of four-chain averages across three dataset/prior-center replicates. The comparison tests whether the endpoint width ordering persists when the step size is halved.

Update the duration if the predefined common-duration fallback is used. Add the actual outcome sentence after analysis. Do not describe this as passing the old continuous-time validation or validating a PI scaling law.

## 6. Reuse the predictive-score result as a table

The completed results report gives positive prior-to-posterior predictive improvements for all 24 targets, approximately 0.055–0.144 nats per held-out point. Reuse the audited source table rather than launching a new predictive experiment. This helps readers distinguish a prior-like hidden norm from a posterior that is wholly unaffected by data.

Define the existing score precisely:

$$\text{NLL}(\mu) = -\frac{1}{n_{\text{test}}} \sum_i \log \mathbb{E}_{\theta \sim \mu} [p_{\theta}(y_i | x_i)], \quad \Delta = \text{NLL}(\gamma) - \text{NLL}(\pi).$$

Average predictive probabilities before taking the logarithm. The mean of the per-state negative log probabilities is a different quantity and must not be substituted. The campaign used all 1,024 test points and 2,048 selected posterior states plus 2,048 independent prior states per target for this check. Verify the actual table’s source counts.

Suggested appendix table: eight architecture/width rows, with columns for median predictive improvement and range across the three replicates, maximum loss  $\widehat{R}$ , minimum loss bulk ESS, and (where S1 is performed) maximum probability  $\widehat{R}$  and minimum probability bulk ESS across the eight fixed points. Retain all 24 per-target rows in a CSV. Reference-pool diagnostics and dynamics diagnostics must be labelled separately if both are included.

If only this predictive summary is missing, recompute it from the existing reference parameter archive in batches. No dynamics sampling is needed. Use the originally specified posterior-state selection and independent prior RNG; do not change the selected sample budget to optimize the gain. Preserve previously audited Monte Carlo errors where available. Across-replicate range is not a replacement estimate of within-target MC error.

## 7. Diagnostics: enough to assess the new claims

### 7.1 Numerical and bookkeeping requirements

For every series used in a supplementary curve:

- Float64, finite values, unique target/chain/transition IDs, correct retained segment.
- Exact matching of data, priors, test indices, model scaling, and step size to its manifest.
- Reject/repeat consistency and correct stored-state timing.
- Unit stride in the preferred route; declared common stride in the limited thinning fallback.
- ACF at lag zero equals 1 to  $10^{-12}$  after normalization; no values outside  $[-1,1]$  beyond  $10^{-10}$ .
- Agreement of FFT and direct-sum ACF on deterministic short fixtures within  $10^{-11}$ .

Check positive-affine and negative-affine invariance of ACF: replacing a nonconstant series by  $ax+b$  for any  $a \neq 0$  must give the same normalized ACF to numerical tolerance. This detects normalization and centering errors. A probability calculated with the wrong logit scaling is a nonlinear change and is not excused by this invariance.

For rejected-state reconstruction, test a sequence with consecutive rejections at the beginning, middle, and end. For step-size comparison, test that  $t = 0.5$  indexes lag 50 at 0.01 and lag 100 at 0.005. Fail on noninteger requested indices rather than silently rounding arbitrary lags.

## 7.2 Per-observable trace diagnostics

Use the project's existing implementation of rank-normalized split/folded  $\widehat{R}$  and bulk/tail ESS, recording its installed version. The rank-based diagnostics assess chain consistency; the plotted ACF uses the raw observable. Modern rank diagnostics and their limitations are discussed by Vehtari et al. [R4]

For S1 require, on the selected four-chain window, for  $V$  and **each of the eight probabilities**:  $\widehat{R} < 1.01$ , bulk ESS at least 1,000, tail ESS at least 400, finite nondegenerate variance, and no obvious sustained drift or stuck interval. For S2 apply these checks to  $V$  at each target/step combination. These are protocol thresholds, not theorem assumptions or guarantees of global convergence.

Inspect first-half and second-half means and ACFs at the reported lags. Export those comparisons; do not force them to agree by trimming a difficult portion after viewing it. A visibly persistent shift prompts a trace-integrity/initialization review. Ordinary noisy ACF tail oscillations do not justify a new run.

Report acceptance and longest rejection streak as algorithm diagnostics. Similar acceptance rates across widths are context, not a proof that accept/reject behavior cannot affect the ACF. A high acceptance rate is not, by itself, evidence of good mixing.

## 7.3 What not to require

Do not estimate an optimal PI or LSI coefficient. Do not integrate an ACF, fit an exponential, extrapolate a zero-lag derivative, or impose a relative-precision gate on a near-zero curve difference. Do not use the old entropy-tilt validity status to screen a probability ACF.

The display bands show between-replicate variation only. With three independent datasets, keep those three outcomes inspectable. Do not turn the chains, held-out points, or millions of transitions into independent experimental replicates.

# 8. Build sequence and acceptance checks

## 8.1 Minimal implementation

Reuse the existing model, sampler, diagnostics, and ACF implementation. Add only:

1. A source adapter exporting the specified probability/loss arrays and manifest.
2. A calculation step that computes per-point, per-chain ACFs, then the prescribed aggregates and paired differences.
3. A rendering step that consumes those fixed tables using the cleanup guide's style.
4. An audit step matching every plotted curve to its source arrays, retained window, and diagnostic row.

`supplement_metrics.py` in this package provides array-only FFT ACF, predictive aggregation, loss aggregation, and exact algorithmic-lag indexing. It deliberately does not load project archives, run samplers, or calculate  $\widehat{R}$ /ESS. Use the project's validated diagnostic package for those. The local

self-check exercises the array calculations with deterministic and synthetic examples; it is not a validation of the actual campaign traces.

## 8.2 Source adapter acceptance

Before exporting all targets, validate one narrow and one wide target in each architecture. Confirm eight correct test indices, exact last-window overlap with the accepted loss export, and repeat consistency. Compare probabilities at saved checkpoints with fresh forward evaluations. Once the adapter is correct, run it across all eligible targets without target-specific formula changes.

The analysis outputs should include:

- `prediction_acf_per_point.csv`: architecture, width, replicate, test index, lag, four-chain-mean ACF.
- `prediction_acf_replicates.csv`: architecture, width, replicate, lag, mean ACF across eight points.
- `prediction_width_differences.csv`: architecture, replicate, test index or mean8, lag, narrow ACF, wide ACF, difference.
- `step_acf_replicates.csv`: width, replicate, step size, integer lag, algorithmic lag, four-chain-mean loss ACF.
- `step_width_differences.csv`: replicate, step size, algorithmic lag, narrow ACF, wide ACF, difference.
- `supplement_diagnostics.csv`: exact observable/window,  $\widehat{R}$ , bulk/tail ESS, acceptance, validity status.
- `recovery_log.json`: reused, forward-recomputed, exactly replayed, or newly sampled for every target/step combination.

Use one row per defined key; duplicates are errors. Tables retain negative ACF values and unfavorable width differences. A plotting function must not filter on the sign of the result.

## 8.3 Publication checks

Verify shared axes, correct color-to-width mapping, explicit shading definition, and readable captions at final paper size. Do not place trace counts or diagnostic badges in the plot. Check that every reported numerical contrast can be reconstructed from the table.

If a numerical eligibility check fails, first distinguish a genuine trace problem from an export bug. Repairing an export bug does not justify changing the posterior or resampling seeds. Mark unresolved cases explicitly in the source tables; do not silently remove the difficult width while presenting a complete comparison.

# 9. If new trajectories are actually necessary

## 9.1 Decide the scope before launching

The **default is zero new posterior sampling**. Search the existing cluster outputs and backups once, check original code/commit and restart metadata, and choose the cheapest valid recovery route. Exact replay is charged to the compute budget even though it recreates an existing realization.

If S1 inputs cannot be recovered cheaply, permit **at most six new target/step-size combinations**, all at  $h = 0.01$ , with four chains each. This is one replacement batch for existing posterior targets. Use only deep endpoint widths 32 and 256 and all three replicates. If only some endpoint combinations are missing, fill those; use matched existing traces for the rest. If the complete all-width S1 export already exists, no new trajectories are needed.

In this fallback, omit the shallow S1 panel and intermediate widths and describe the actual deep endpoint-only scope. Do not rerun all 24 targets just to recreate missing probability logs. S2 remains reuse-only; no new  $h = 0.005$  trajectories or expensive replay are authorized by this revised plan. If the optional inputs are missing, omit S2.

Budget and numerical adequacy determine whether a supplement is performed, never the sign of its width effect. A missing supplementary result is preferable to an open-ended campaign.

Do not launch new reference chains: the original report says all 24 reference targets passed. Retrieve their saved endpoint states. If those states or the exact target configuration are lost, the cheap fallback premise no longer holds; stop rather than regenerate a new posterior campaign under an assumed configuration.

## 9.2 Exact replacement-run settings

For each missing deep endpoint S1 combination at  $h = 0.01$ :

1. Load its existing dataset, test set, prior centers, model configuration, and sampler commit. Check hashes before state initialization.
2. Initialize four dynamics chains from the corresponding four audited reference-chain endpoints. Use fresh deterministic streams labelled by target ID,  $h$ , chain ID, and the suffix `supplement_recovery_v1`. Derive seeds with SHA-256 over canonical strings; do not use Python's `process-randomized hash()`.
3. Use  $h = 0.01$  with no adaptation or retuning. Discard 20 units of algorithmic duration, or 2,000 transitions.
4. Retain **102,400 transitions per chain**, recording  $V$ , all eight probabilities, acceptance flags, and transition/evaluation counters at every step. Save exact state and RNG checkpoints every 2,048 transitions. Rejections produce repeated scalar values.
5. S1 uses all 102,400 retained transitions. Any S2 analysis uses existing trajectories only and its separate matched-window rules.
6. Apply the relevant Section 7 diagnostics. No new run is triggered merely because the width effect is weak, absent, or adverse.

If a stochastic diagnostic fails and there is no deterministic bug, allow **one continuation of 102,400 transitions per chain** for that combination. Preserve the failed first segment. Reapply diagnostics to the prescribed final window, keeping its length matched across widths and steps. Longer already-existing traces may be reused in the same way. Record the shifted window explicitly. Do not present a selectively extended high-precision curve beside an unreported short comparator, and do not enlarge the scientific comparison after viewing the effect.

For S1, the preferred uniform window is fixed at 102,400. An extension used to resolve initialization/diagnostic issues contributes its final 102,400, with the window shift disclosed; it does not reduce ACF Monte Carlo error by pretending the window became longer. For S2, use only available eligible matched windows. A failed S2 diagnostic does not trigger replacement sampling in this revised plan.

Stop at two retained blocks per new chain. Maximum total transitions for six  $h = 0.01$  combinations are **2,505,600** for the initial batch (including 2,000 discard steps per chain), or **4,963,200** if every chain uses its single continuation. Exact replay counts against the same practical work budget. No wall-clock promise is possible without a benchmark on the user's hardware.

## 9.3 Runtime and storage preflight

Benchmark 200 transitions of the exact implementation at deep width 256 with all eight probability outputs enabled. Synchronize the accelerator before and after timing; include scalar-recording overhead and record both total gradient evaluations and wall time. Estimate the batch using the

actual transition count and measured throughput; also account for simultaneous-chain memory. Benchmarking does not change scientific parameters.

Eight probability traces require about 26.2 MB per four-chain target at 102,400 draws; loss, flags, and counters add modest storage. Full parameter history is not needed. The batch is dominated by network/gradient evaluations, not ACF computation.

If the preflight is unaffordable, do the reuse-only subset. Do not reduce the number of replicates after seeing which replicate is slow or unfavorable, change  $n$ , or switch to minibatch gradients to fit the budget.

## 10. Final paper placement and stopping rule

Complete the revised main spectral quantile/coverage analysis and threshold-count table first, while preserving the accepted loss-ACF analysis. Put S1 in the appendix and reference it in the paragraph discussing loss decorrelation. Put S2 next to the sampler description or diagnostic appendix and reference it once when stating the fixed-kernel scope. Place the already-completed predictive check in the compact diagnostic table.

If only one supplement is affordable, prefer **S1**, because it adds a genuinely different observable relevant to prediction. Prefer S2 next only when its existing eligible trajectories are accessible without an expensive replay or new sampling. Do not add an extra figure solely to restate ESS, acceptance, or the same spectral quantiles in another format.

Stop when the selected figures, paired-difference tables, and source audits are complete. The stopping rule is completion and numerical adequacy, not a favorable narrative. Flat predictive curves or step-sensitive width differences are reported as such; they do not authorize searching additional observables or new seeds.

## References

- [R1] Pavel Izmailov et al. What Are Bayesian Neural Network Posteriors Really Like? ICML 2021, PMLR 139. Motivates examining predictions and distinguishing posterior approximation from predictive quality. <https://proceedings.mlr.press/v139/izmailov21a.html>
- [R2] Mrinank Sharma et al. Do Bayesian Neural Networks Need To Be Fully Stochastic? AISTATS 2023, PMLR 206. Figure 2 examines chain behavior in function space. Our fixed-point autocorrelation experiment is a different, narrower measurement. <https://proceedings.mlr.press/v206/sharma23a.html>
- [R3] Original local source: `execution_runbook.md`, Sections 2, 5, 7, and 8; `campaign.yaml`; supplied results report `Pasted markdown(20261001-203049).md`. These define the exact targets, prescribed stored probabilities, sampler steps, and completed main-result diagnostics. Archive availability still requires inspection.
- [R4] Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-Normalization, Folding, and Localization: An Improved  $\hat{R}$  for Assessing Convergence of MCMC (with Discussion). *Bayesian Analysis* 16(2), 2021. DOI: 10.1214/20-BA1221. Thresholds in this protocol are chosen for this campaign. <https://arxiv.org/abs/1903.08008>